Cloud CI/CD Pipeline for a Java Web Application

Technical report | DevOps CIA 2
Mohit | 5 November 2026

Problem and implemented solution

Manual builds and deployments are difficult to repeat reliably. This project automates the path from a Git commit to a browser-accessible Java application on AWS. A task tracker provides add, complete and delete operations while the pipeline demonstrates compilation, testing, container publishing and Kubernetes deployment.

A push to main has been verified to deploy version 1.0.1 automatically. The release passed nine automated tests, published a new ECR image and replaced the running Kubernetes pod.



Ansible applies the desired Deployment, Service and Ingress configuration.

Browser :80 -> Traefik Ingress -> ClusterIP Service :80 -> Java Pod :8080

Tool selection

Tool	Reason for selection
Java 21 and Spring Boot	Java web application with embedded server, validation and Actuator health endpoints.
Maven and GitHub Actions	Reproducible build and tests; automated jobs triggered by source changes.
Docker and Amazon ECR	Portable Java runtime image and a private AWS container registry.
EC2 and K3s	Small Ubuntu server hosting a lightweight Kubernetes cluster.
Ansible and Traefik	Repeatable server/deployment configuration and HTTP ingress routing.

Pipeline stages and artifacts

The workflow contains two dependent jobs. CPU-intensive Maven work runs on GitHub. The deployment job runs on EC2 only after the build and tests succeed. This keeps the 2 GiB instance focused on packaging and running the application.

Stage	Purpose and result
1 Checkout	Obtain the source revision associated with the push to main.
2 Build and test	Select Java 21 and run ./mvnw -B -ntp clean verify. A failing test prevents deployment.
3 Preserve artifacts	Upload the tested executable JAR and Surefire test reports to the workflow run.
4 Package image	Download that same JAR on EC2. Dockerfile.runtime adds the Java 21 JRE and runs the app as a non-root user.
5 Publish to ECR	Authenticate using the EC2 role and push a tag containing commit SHA, run ID and attempt. Tags are never reused.
6 Apply configuration	Ansible refreshes ECR credentials, renders the new image into Kubernetes YAML and applies the release.
7 Verify	Wait for rollout completion, check Actuator health through Traefik, and verify the visible application version.

Why there are two Dockerfiles

The original multi-stage Dockerfile builds and tests from source for local use. Dockerfile.runtime packages the exact JAR already tested in CI. This avoids a second Maven build on the small EC2 server and makes the deployed artifact traceable to the test job.

Trigger and failure behavior

Application/configuration pushes to main and manual workflow runs are supported. Documentation-only changes do not redeploy. Deployments are serialized. Pull-request triggers are intentionally absent because the deployment runner has administrator privileges. Failed tests leave the current deployment running; a failed rollout makes the run fail and requires diagnosis or rollback.

Repository

github.com/MohithR2005/devops_final_lab

Cloud and Kubernetes deployment

Resource	Configuration
Region and server	Sydney ap-southeast-2; Ubuntu 24.04; t3.small; 20 GiB root volume.
Instance	i-009c142441c60c7a8, named final_lab_server.
Namespace and application	final-lab namespace, Deployment, ClusterIP Service and Traefik Ingress.
Runtime	One pod, Java port 8080; Service port 80; public HTTP through EC2 port 80.
Resources and probes	100m CPU / 256Mi request; 1 CPU / 512Mi limit; startup, readiness and liveness probes.
Deployment strategy	Recreate avoids overlapping Java pods; updates have brief downtime.

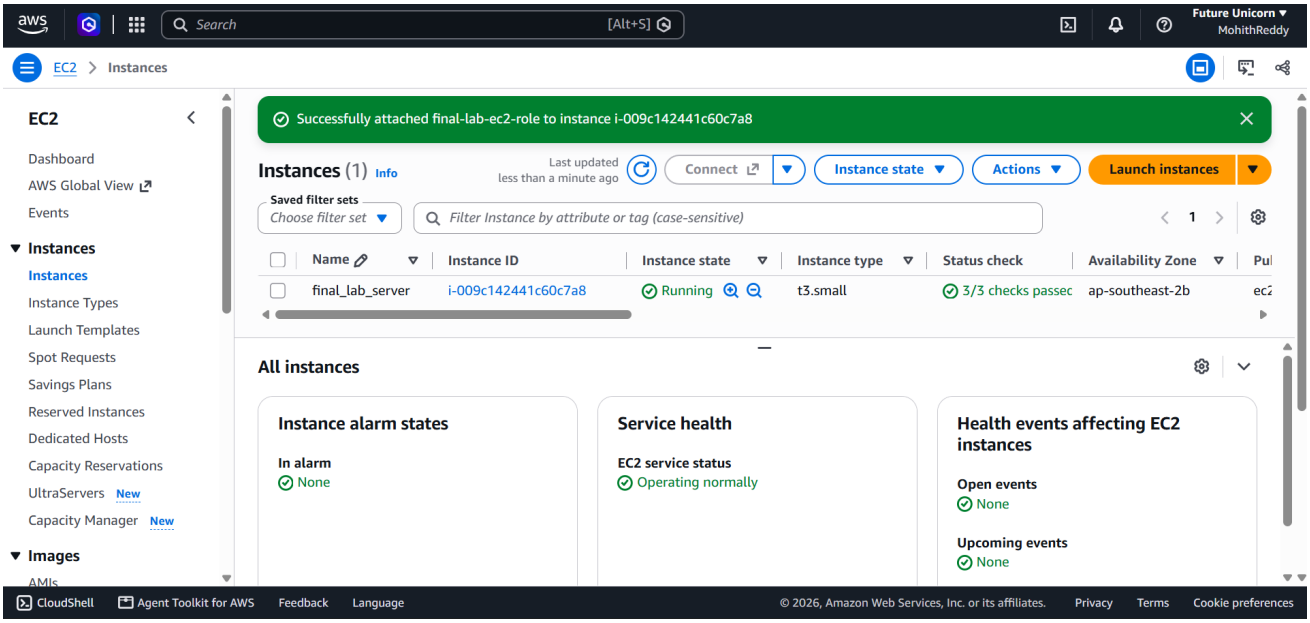


Figure 1. EC2 console evidence captured during setup: the t3.small instance is running and its IAM role is attached. Current resource details are preserved in cloud-resources.json.

Credential lifecycle

The EC2 instance profile supplies temporary AWS credentials. Ansible generates ecr-secret in the final-lab namespace and installs a systemd timer that refreshes it every six hours. It also refreshes credentials before each deployment. ECR tokens last twelve hours; renewal prevents the next image pull from failing after a token expires.

No public Kubernetes API or SSH connection is required by the pipeline. The runner connects outbound to GitHub. K3s, the runner and the renewal timer persist as system services across SSH disconnections.

Testing and successful execution

The verification suite contains service unit tests, real HTTP integration tests and an application-context test. The version assertion reads app.version dynamically, so a version-only demonstration remains a valid test of the configured application.

Test suite	Tests	Failures	Errors
TaskWebTest	4	0	0
TasktrackerApplicationTests	1	0	0
TaskServiceTest	4	0	0
Total	9	0	0

Coverage includes trimmed/invalid titles, task lifecycle, missing IDs, HTML escaping, page rendering, static CSS and the health endpoint. Surefire XML reports were downloaded from the successful run rather than inferred from a screenshot.

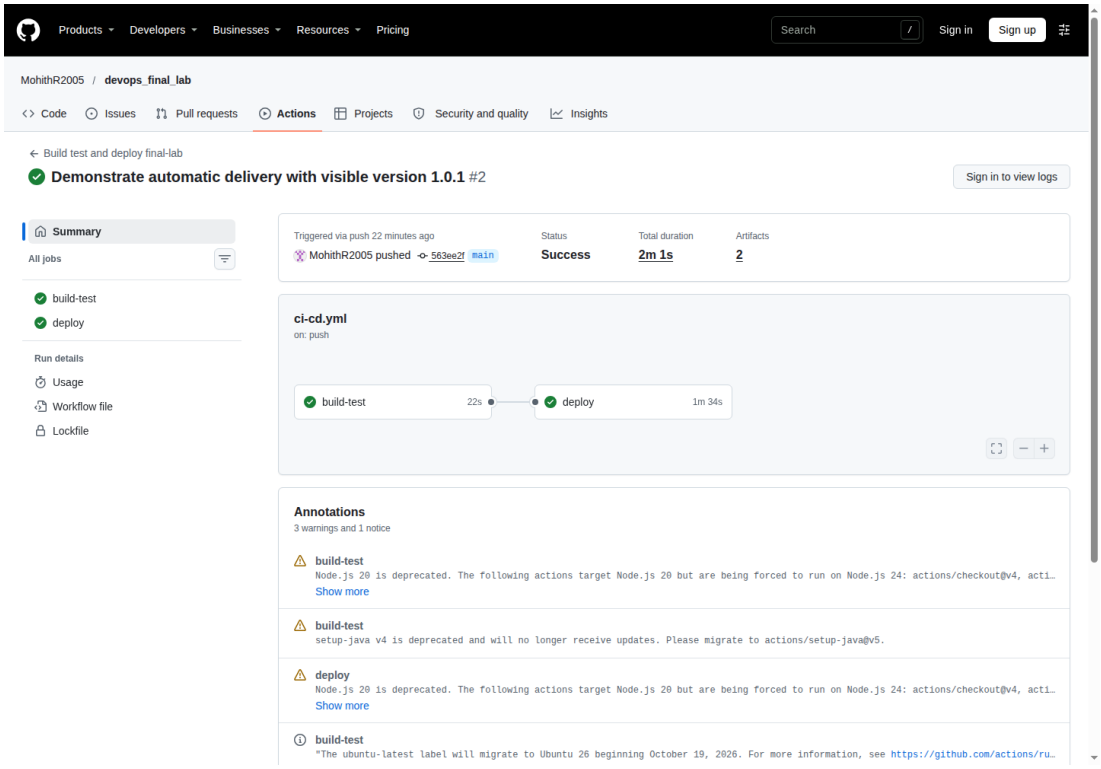


Figure 2. Successful GitHub Actions run for the version-change commit. Both build-test and deploy completed successfully.

Verified execution

Run: 42074750277
Commit: 563ee2fff28b61a817c10de1611d4b5bf9ba58f5
Result: success; 9 tests, 0 failures, 0 errors.
Open the verified workflow run

Change and redeployment demonstration

A separate commit changed app.version from 1.0.0 to 1.0.1 in application.properties. Pushing that commit automatically started a new workflow. The pipeline tested and packaged the new source, pushed a new image and applied the updated Kubernetes Deployment.

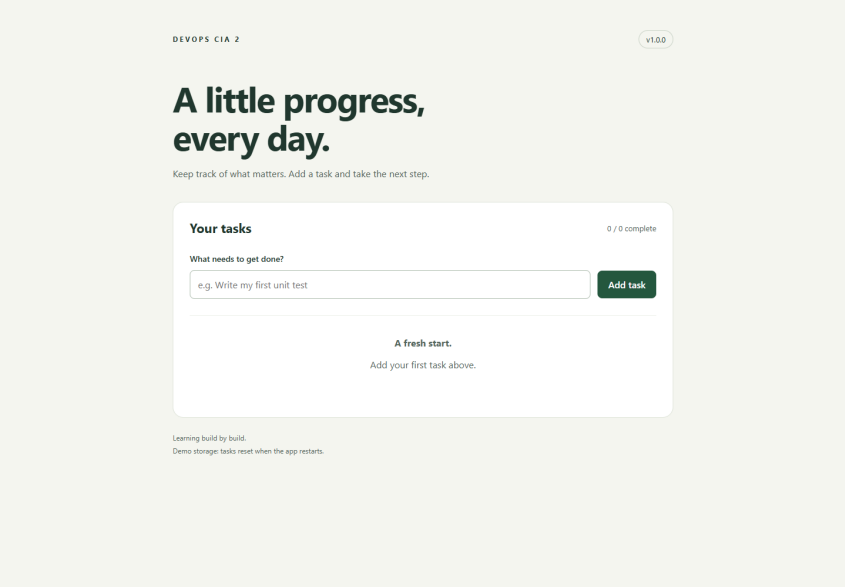


Figure 3. Browser capture before the demonstration change, showing the version 1.0.0 badge.

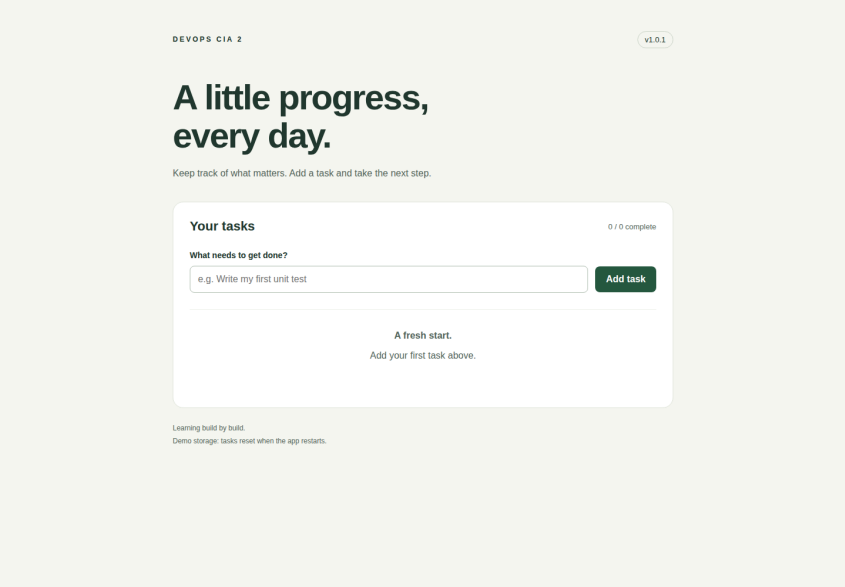


Figure 4. Browser capture after the automatic deployment, showing version 1.0.1 at the same endpoint.

Release traceability

The ECR tag includes the complete Git commit hash plus the workflow run ID and attempt. Kubernetes references that tag. The report evidence folder contains the image digest, running resource output, run metadata and test summary, allowing the commit-to-image-to-deployment chain to be checked.

Challenges security and limitations

Challenge	Resolution
Maven command unavailable	Used the checked-in Maven Wrapper and a compatible JDK.
Windows ECR login helper error	Published the initial image using an isolated temporary Docker configuration; CI uses Linux Docker login.
SSH timeout and changing hotspot IP	Restricted SSH to the actual current public IP. Systems Manager was used for subsequent setup.
Windows private-key permissions	Restricted the PEM file ACL so OpenSSH accepted it. The key is not in GitHub.
OIDC setup denied by AWS	Organization policy blocked provider creation. Used the approved EC2 runner and existing instance role.
Small instance resources	Hosted Maven build; lightweight runtime image; one Java pod and bounded memory.
Expiring ECR authentication	Automated credential refresh every six hours and immediately before deployment.

Security controls

No static AWS access key, PEM file or Kubernetes Secret is committed. ECR image publishing permissions are scoped to final-lab. The image runs as a non-root application user. The pipeline uses a read-only GitHub token, serializes deployments and does not execute pull-request code on the deployment runner.

Limitations and future improvements

This is an assignment deployment, not a production service. It has one node, one replica, HTTP without TLS, shared in-memory tasks and no login or CSRF protection. Tasks disappear on pod restart. The runner has Docker access and passwordless sudo, so trusted main-branch write access is essential. There is no automatic rollback. Production improvements include a database, HTTPS, authentication, isolated ephemeral runners, stronger workload permissions and a highly available cluster.

Cost and lifecycle

EC2 must remain running for the demonstration and deployment jobs. Stop it after evaluation when it is no longer needed, while remembering that stored volumes and images can still cost money. A stop/start may change the public IP. Do not terminate the instance before the demonstration.

Presentation and submission checklist

Demonstrate in this order

1. Open the live application and demonstrate adding, completing and deleting a task.
2. Explain the workflow and show the latest green Actions run.
3. Show ECR images and Kubernetes pod/Service/Ingress evidence.
4. Edit app.version on GitHub from 1.0.1 to 1.0.2 and commit to main.
5. Watch build-test and deploy finish. Refresh the website and show the new version.

Explain the architecture in one sentence

“GitHub tests my Java application with Maven, passes the tested JAR to the EC2 runner, and Docker, ECR, Ansible and Kubernetes deliver it automatically to the website.”

Submission contents

Deliverable	Location
Source, Maven and tests	src/, pom.xml and Maven Wrapper files
Container build	Dockerfile, Dockerfile.runtime and .dockerignore
Pipeline	.github/workflows/ci-cd.yml
Deployment and automation	k8s/, ansible/ and scripts/refresh-ecr.sh
Evidence	docs/evidence/ with browser screenshots and verified JSON/text outputs
Report and preparation	This report, DEMO-CHEATSHEET.pdf and DEMO-AND-VIVA.md

References

GitHub Actions runner documentation
docs.github.com/en/actions/how-tos/manage-runners/self-hosted-runners/add-runners

Amazon ECR registry authentication
docs.aws.amazon.com/AmazonECR/latest/userguide/registry_auth.html

K3s documentation: docs.k3s.io
Ansible playbooks: docs.ansible.com